

CSE47201 Computer Vision Assignment 2 [100 points]

Deadline : 2025. 12. 21 (Sun) 23:59

- If you have any questions, please post it on the blackboard/discussion/'assignment 2' board.
- You can write reports either in Korean or English.

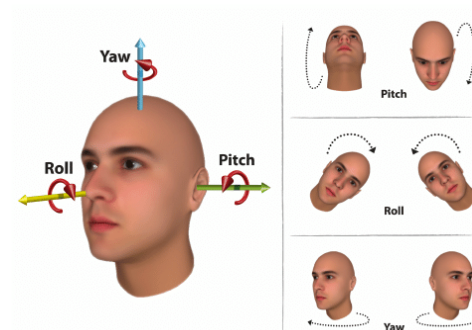
1. **[Implementing facial segmentation and head pose estimation 45 points]** Please Download the facial database and scripts using the link below:

<https://drive.google.com/file/d/1mVcGua0TfuVAjduMOY1c4HcoMmS3WVoB/view?usp=sharing>

- The data structure is as follows.
 - `model.py`
 - `dataset.py`
 - `inference.py`
 - `data.csv`
 - `weights.pth`
 - `train/`
 - `images/*.jpg`
 - `masks/*.png`
 - `test/`
 - `images/*.jpg`
- The dataset is organized into two subsets: train and test. Each subset contains an `images` folder with raw images in JPEG format (*.jpg), and a corresponding `masks` folder that stores pixel-wise annotation masks in PNG format (*.png). The annotation masks are provided separately for each semantic class (e.g., *hair*, *nose*, *skin*) and follow a naming convention aligned with the corresponding image identifiers.
- You are provided with a CSV file named '**data.csv**', which contains the following columns: **Yaw**, **Pitch**, and **Roll**. The **Yaw**, **Pitch**, and **Roll** columns represent facial orientation angles. Each of these values ranges from **-90 to 90** degrees.

Yaw	Pitch	Roll
5.359253	-0.239891	-0.564011
-5.858994	-16.367280	0.309814
-0.683464	-15.296036	1.553177
4.465645	-6.217133	-0.618805
21.726631	-14.743744	-4.550049
...	...	...
24.259483	-7.338150	-3.581505
-16.914520	-0.953232	-1.116104
-5.050949	-8.896774	-0.317261
1.673561	-8.934494	-0.375473
-44.276080	-18.463852	8.781601

Yaw, Pitch, and Roll are values that represent the orientation of the face as rotational angles in 3D space. **Yaw** indicates the degree of horizontal rotation of the face, changing when the head turns left or right. **Pitch** represents the vertical tilt of the face, changing when the head moves up or down. **Roll** denotes the sideways tilt of the face, changing when the head leans toward either shoulder.



(Enhanced real-time head pose estimation system for mobile device - Scientific Figure on ResearchGate.

Available from:

https://www.researchgate.net/figure/Orientation-of-the-head-in-terms-of-pitch-roll-and-yaw-movements-describing-the-three_fig1_279291928 [accessed 8 May 2025])

- There are 3 python scripts(model.py, dataset.py, inference.py).
- Use the database when training your model while using the testing database when evaluating your model.
- Please upload a .zip file (Assignment2-1.zip) and unzip that file in your local colab environment.
- Please confirm the folder path and settings in *python* scripts.

```
[1] 1 from google.colab import drive
    2 drive.mount('/content/drive')
    3
    4 import os
    5 os.chdir('/content/drive/path_to_your_workspace')
```

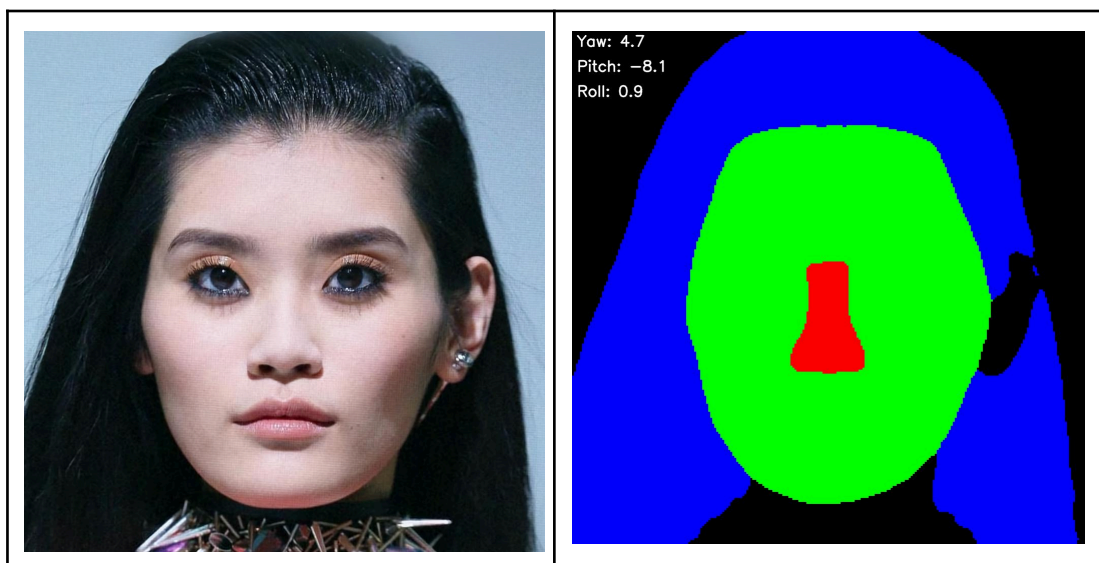
↻ Drive already mounted at /content/drive; to attempt to forcibly remount,

When you execute the *inference.py*, you can infer the example network that we provide whose model structure is defined in *model.py* and whose weight is saved in the *weights.pth*. (The provided *weights.pth* file is from an incomplete training process.)

```
1 !python inference.py --image test/images --weights weights.pth --save output
```

(example command for executing *inference.py*)

Below are examples for human images and corresponding color-coded segmentation masks obtained from *inference.py* for the visualization having 3 classes (Hair: blue, RGB=(0, 0, 255), Skin: green, RGB=(0, 255, 0), Nose: red, RGB=(255, 0, 0), Background: black, RGB=(0, 0, 0)):



Please implement the training code and the loss class in *train.py*, and proceed with model training. After completing the training, save the model weights file (*.pth). Try running *train.py* to train the model, and use *inference.py* to perform segmentation and head pose estimation on the test images (*.jpg) contained in the test folder.

[35 points] Please Implement the 'train.py' for training your own model. If you want to, you can also modify the model in 'model.py':

- a. You may use the cross-entropy loss to make the predicted segmentation mask close to the ground-truth segmentation mask.
- b. You may use mean squared error (MSE) loss when training pose estimation.

[10 points] Train your own head segmentation and pose estimator using the code and run your trained network on the 10 test images and visualize the obtained results for the test images contained in the test folder. (To get the full score, your results need to have reasonable quality to capture human face segmentation and angles. Especially, we encourage the average error for yaw, pitch, roll angles to be within 10 degrees, to get the full score. Please properly tune the training stage.)

2. **[Implementing facial generation network 45 points]** Please download the data using the link below:

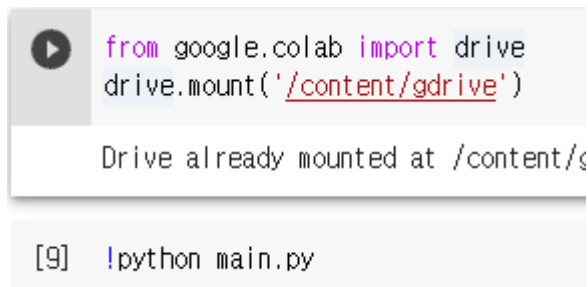
<https://drive.google.com/drive/folders/1--Dtsk9-j5V0Zo4FnnLkxZBwILfBks2C?usp=sharing>

- Please use the data in the 'train' folder as the training data when training your model, while using the data in the 'test' folder as the testing data for evaluating your model.
- Please download the network architecture, *.py and weight file(*.pth), contained in the link below:

https://drive.google.com/drive/folders/11olfcJa5cc_L8K3yh-zT9ZnaDMTusSdt?usp=sharing

We provided the 'Tester' class in main.py that initializes the facial generation network with the weights '*.pth' provided. Please do the following things for this problem.

- Please upload datasets in your own environment (google drive is recommended).
- Please upload .pth files in your own environment (google drive is recommended).
- Please upload .py files in your local colab environment.
- Please confirm the folder path and settings in main.py.



```

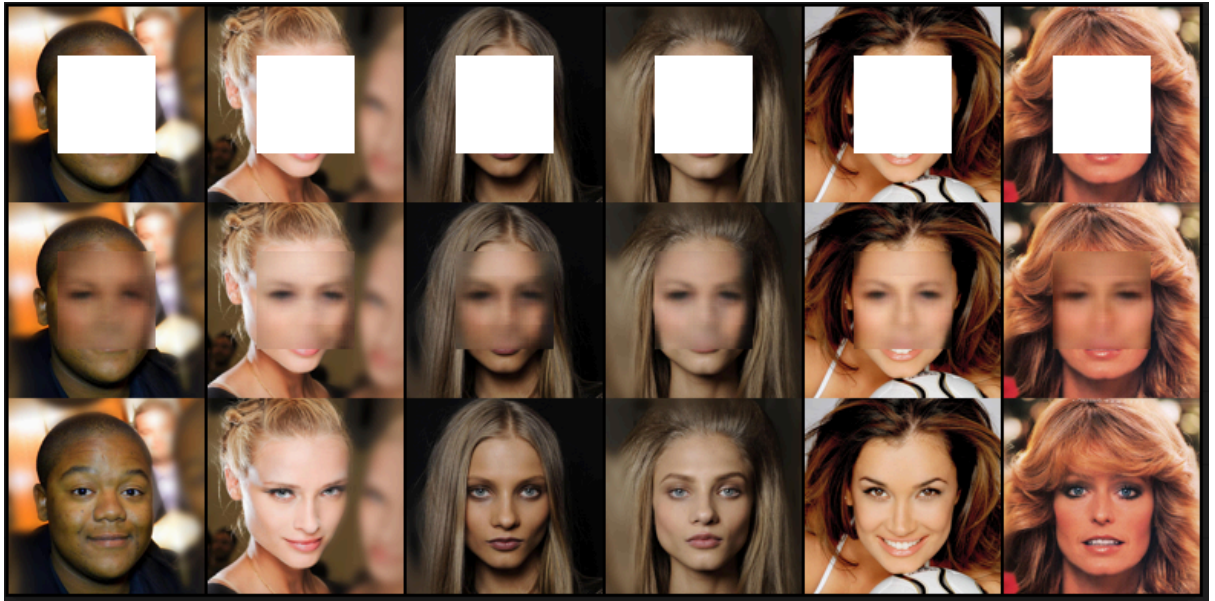
from google.colab import drive
drive.mount('/content/gdrive')

Drive already mounted at /content/gdrive

[9] !python main.py

```

- You can use above codes for mounting the google drive and running the main.py. If you set all the things correctly, you can obtain the visualization in 'test.png' as below, for erased image, reconstructed image and original images in each row.



'Trainer' class is reserved for training your context encoder on the training data provided, when called from the main() function as follows:

```
trainer = Trainer(epochs, batchSize, learningRate)
trainer.train()
```

- Output of the network is the original image in the 'train' folder and input is the erased image.
- Please use the generator architecture implemented in 'model.py'. Also, you can use discriminator architecture in the same file, if necessary.
- Please use the dataloader implemented in FaceDataset class. It returns random-erased images for training (via 'apply_random_mask' method) while returns center-erased images for testing (via 'apply_center_mask' method).

[35 points] Please implement the Trainer class including the train() function properly to train the network. You may require involving both L2 losses and adversarial loss to make the network properly estimate the facial regions. **You can get the full score using only L2 loss**, while if you are further interested in involving the adversarial loss, you can also refer [1] for the implementation.

[1] Context Encoders: Feature Learning by Inpainting, CVPR'16.

[10 points] Please visualize and analyze the generated images using the Tester class but using your own trained model.

- [10 points]** Please note and apply the below details.

- Send all files(.zip) via **Blackboard**.

- b. In **20241111_SeungryulBaek_cv_ass2.zip** file, you need to have python files (*.py), pre-trained network weights (.pth) and a report as follows:

<Example>

```
20241111_SeungryulBaek_cv_ass1.zip
|--Prob1.py
|--Prob1.pth
|--Prob2.py
|--Prob2.pth
--Report.pdf
```

- c. You need to write a report within 10 pages, properly summarizing your solutions or results for each problem.
- d. Code should have some **comment** that increase readability(it is also grading points)
- e. Your pre-trained weights in .pth need to be properly loaded.

<Tip>

Maximum keeping time for one session in the CoLab is 12 hours when you use GPU mode, but if you remain in your CoLab session without typing anything over 90 mins, the session would be automatically shut down. So, please refer below links to prevent it.

<https://naenjun.tistory.com/18>

<https://stackoverflow.com/questions/57113226/how-to-prevent-google-colab-from-disconnecting>